7.5 Adaptive Control of a Real DC Motor

Example 7.6 of the reader cites the experiment by Abram and Fieg, where the adaptive controller is implemented on an Arduino Due [1, 2]. The present section realises the same self-tuning pole-placement idea on an own test rig. The hardware consists of an ESP32, an L298N H-bridge, and a DC motor with a quadrature encoder. The controller is not hand-coded on the microcontroller; instead, the real-time code is generated automatically from the Simulink model.

The structure follows the indirect adaptive-control scheme of Figure 7.1 in the reader. The plant parameters are estimated recursively, the controller is redesigned from these estimates, and the resulting incremental PI algorithm is executed on the ESP32 input/output interface. Figure 7.2 shows the corresponding arrangement for the motor rig.

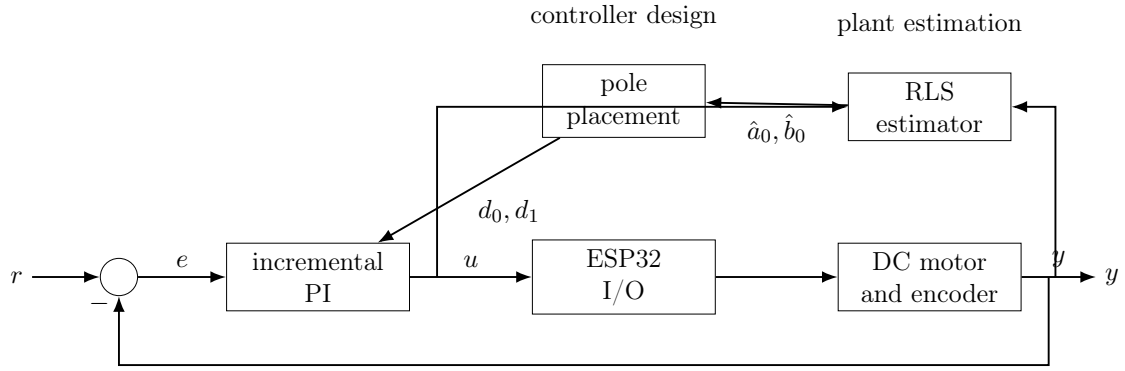

Figure 7.2: Self-tuning regulator used for the ESP32 DC motor rig.

Example 7.7: Numerical constants for the ESP32 motor rig

The sampled controller uses the period $T = 0.08\,\mathrm{s}$. The estimator is initialised with $P(0) = 100I$ and $x_e(0) = \begin{bmatrix} -0.5 & 1.0 \end{bmatrix}^T$. The forgetting factor used for the validated run is $\alpha = 0.98$, the desired polynomial is $q(z) = z^2 - 0.5z + 0.06$, and the motor command is saturated at ± 255. On the hardware, the PWM output is GPIO 14, the two H-bridge direction inputs are GPIO 26 and GPIO 27, and the quadrature encoder signals are read on GPIO 32 and GPIO 33.

7.5.1 Plant model

The motor is modelled by the first-order armature-to-speed relation used in the reader. Electrical dynamics are neglected, so that the inertia J, viscous friction coefficient b, motor constant K_m, input u, and angular speed ω satisfy

$$J\dot{\omega} = -b\omega + K_m u. \tag{7.17}$$

The corresponding continuous-time transfer function is

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{K_m/J}{s + b/J}. \tag{7.18}$$

With zero-order hold sampling and sampling period $T = 0.08\,\mathrm{s}$, the discrete transfer function is written in the notation of the reader as

$$G(z) = \frac{b_0}{z + a_0}, \tag{7.19}$$

where

$$a_0 = -\exp\left(-\frac{bT}{J}\right), \qquad b_0 = \frac{K_m}{b}\left(1 - \exp\left(-\frac{bT}{J}\right)\right). \tag{7.20}$$

The identified parameter vector is therefore

$$x_e = \begin{bmatrix} a_0 & b_0 \end{bmatrix}^T. \tag{7.21}$$

7.5.2 Recursive least squares

The discrete plant model in (7.19) gives the one-step regression

$$y(k) = C(k)x_e(k), \qquad C(k) = \begin{bmatrix} -y(k-1) & u(k-1) \end{bmatrix}. \tag{7.22}$$

The recursive least-squares update with forgetting factor α is

$$L(k) = \frac{P(k-1)C^T(k)}{C(k)P(k-1)C^T(k) + \alpha}, \tag{7.23}$$

$$x_e(k) = x_e(k-1) + L(k)\left(y(k) - C(k)x_e(k-1)\right), \tag{7.24}$$

$$P(k) = \frac{\left(I - L(k)C(k)\right)P(k-1)}{\alpha}. \tag{7.25}$$

The initial values are

$$P(0) = 100I, \qquad x_e(0) = \begin{bmatrix} -0.5 & 1.0 \end{bmatrix}^T. \tag{7.26}$$

Two implementation details are added to the textbook recursion. First, the covariance update is suspended when the input/output data do not contain excitation. Without this excitation gate, the division by $\alpha < 1$ in (7.25) increases P without bound while the motor is at rest. This covariance windup makes the subsequent parameter transient too aggressive and can lead to bursting. Secondly, the controller-design step uses a guard $|b_0| > 10^{-3}$. If the estimated numerator approaches zero, the pole-placement formula would otherwise divide by a nearly singular plant gain.

7.5.3 Pole placement and PI control

The desired closed-loop polynomial is chosen as

$$q(z) = z^2 - 0.5z + 0.06 = (z - 0.3)(z - 0.2). \tag{7.27}$$

For $T = 0.08\,\mathrm{s}$, these discrete poles correspond to continuous poles close to $-15\,\mathrm{s}^{-1}$ and $-20\,\mathrm{s}^{-1}$. The target settling time is therefore about 250 ms.

Writing $q(z) = z^2 + q_1 z + q_0$, the controller parameters are computed from the current plant estimates by

$$d_0 = \frac{q_0 + a_0}{b_0}, \qquad d_1 = \frac{q_1 + 1 - a_0}{b_0}. \tag{7.28}$$

The implemented controller is the incremental PI law

$$u(k) = u(k-1) + d_1 e(k) + d_0 e(k-1), \tag{7.29}$$

where $e(k) = r(k) - y(k)$. The command is saturated at ± 255. The saturated value, not the unsaturated internal value, is fed back as $u(k-1)$ in (7.29); this is the anti-windup mechanism used in the implementation.

The transient cannot be shaped by the pole locations alone. The closed-loop transfer function contains the controller zero

$$z_0 = -\frac{d_0}{d_1}. \tag{7.30}$$

This is the same structural effect described in the reader as a phase error caused by the different numerator order. A one-degree-of-freedom controller does not provide an independent numerator design, and therefore cannot remove this zero without changing the controller structure.

7.5.4 Simulation

The validation run uses the normalised plant $K_m = 1$, $b = 1$. The inertia switches from the small flywheel to the large flywheel and back to the small flywheel, abbreviated as S–L–S, with

$$J = 1 \qquad \text{and} \qquad J = 24. \tag{7.31}$$

The ratio 24:1 corresponds to the two flywheels. The run lasts 96 s. The reference is a rectangular speed command with amplitude 50 rad/s and period 16 s, starting at $t = 4$ s from standstill. Measurement noise is included.

With $\alpha = 0.98$, the overshoot before the inertia change lies between 0.74 % and 0.84 %. The 2% settling time is 0.40 s. After the L–S change the first transient reaches 1.83 % overshoot and settles in 0.56 s. The next two transients give 0.78 % with 0.48 s, and 0.77 % with 0.40 s, respectively.

The motor command remains inside the saturation limits ±255. During standstill the trace of P remains constant at 200, showing that the excitation gate prevents covariance windup. The maximum trace over the whole run is also 200. After the S–L inertia change, b_0 reconverges after 12.24 s, or 8.24 s after the first exciting reference edge. After the L–S change, the command sign changes at 0.12 times per second; this does not indicate a limit cycle.

The forgetting factor is deliberately set to $\alpha = 0.98$, rather than $\alpha = 1.0$. With $\alpha = 1.0$, the covariance becomes too small during the earlier part of the run. After the L–S inertia change the estimator then does not relearn the new plant quickly enough, and the closed loop ends in a limit cycle with command chattering.

7.5.5 Second-order plant

7.5.6 Hardware realisation

The hardware platform is an ESP32-WROOM module connected to an L298N H-bridge. The L298N introduces an approximate voltage drop of 4 V, so with a 12 V supply the motor sees about 8 V. The motor is equipped with a quadrature encoder. The sampling time is $T = 0.08$ s.

The controller is implemented by automatic code generation from Simulink using the Embedded Real-Time (ERT) target and the ESP32 target support. External Mode is used for Monitor and Tune operation during experiments.

The closed loop runs on the hardware. However, the encoder resolution in counts per revolution has not yet been calibrated. All reported speeds on the hardware are therefore in gain scale. Depending on whether the encoder is interpreted as 11 or 44 impulses per revolution, the scale differs by a factor of 4. No absolute speed values in rad/s are treated as verified, and no hardware settling times are stated here.

Table 7.1: ESP32 pin assignment for the DC motor test rig.

Signal	ESP32 pin
PWM	GPIO 14
IN1	GPIO 26
IN2	GPIO 27
Encoder A	GPIO 32
Encoder B	GPIO 33

7.5.7 Conclusion

The first-order self-tuning regulator from the reader was transferred to an ESP32-based DC motor rig. In simulation, the excitation gate and the $|b_0| > 10^{-3}$ guard keep the indirect adaptive loop well behaved during standstill and inertia changes. The validated run shows bounded command effort, fast reconvergence of the plant numerator after the S–L change, and no limit-cycle behaviour after the L–S change when $\alpha = 0.98$ is used.

The hardware implementation is operational, but quantitative speed results remain limited by the uncalibrated encoder gain. The next step is therefore encoder calibration before absolute speed plots and hardware settling times can be reported.

Bibliography

[1] A. Ravazzolo-Mehrle, *Advanced Control Engineering 2 - Optimal Control.* MCI Innsbruck, 2026. Reader, spring term 2026.

[2] Abram and Fieg, "Adaptive control experiment with an arduino due." cited in [Reader].

[3] K. J. Aström and B. Wittenmark, *Adaptive Control.* Addison-Wesley, 2 ed., 1995.

[4] L. Ljung, *System Identification - Theory for the User.* Prentice Hall, 2 ed., 1999.